A1ELOS GLOBAL NUMEROLOGY

ARQUIVO DE CONFIGURAÇÃO DO SERVIDOR (MAIN.PY)

Versão Consolidada, Corrigida e Pronta para Deploy
02 de agosto de 2026

---

1. Instruções de Implementação

Este documento contém o código-fonte integral do arquivo **main.py**. A estrutura foi reorganizada para sanar o erro de execução no Render (*NameError: name 'app' is not defined*), garantindo que a instância do FastAPI seja criada antes da definição de qualquer rota ou montagem de arquivos estáticos. Para implementar, substitua todo o conteúdo do arquivo atual no seu repositório por este código.

```
# -*- coding: utf-8 -*-
# main.py - Global Numerology API (Versão Corrigida 02/08/2026)

import os, json, uuid, logging
from datetime import date, datetime
from typing import Optional, List
import stripe
from fastapi import FastAPI, HTTPException, Request, Body
from fastapi.responses import RedirectResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
import secrets, string
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

#
# 2. CONFIGURAÇÃO GLOBAL
#
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

stripe.api_key = os.getenv(&quot;STRIPE_SECRET_KEY&quot;)
STRIPE_PUB = os.getenv(&quot;STRIPE_PUBLISHABLE_KEY&quot;, &quot;&quot;)
SENDGRID_KEY = os.getenv(&quot;SENDGRID_API_KEY&quot;)
FROM_EMAIL = os.getenv(&quot;FROM_EMAIL&quot;, &quot;noreply@a1elos.com.br&quot;)
SITE_URL = os.getenv(&quot;SITE_URL&quot;, &quot;https://global-numerology.onrender.com&
ADMIN_EMAIL = &quot;arvigne@gmail.com&quot;
```

```
#
# 3. DICIONÁRIOS DE PREÇOS E PRODUTOS (STRIPE)
#
# IDs de Preços do Stripe (Price IDs)
PRICES = {
    &quot;de&quot;: {&quot;express&quot;: &quot;price_1PZ9X9L0v2o0&quot;, &quot;completo
    &quot;ar&quot;: {&quot;express&quot;: &quot;price_1PZ9XAL0v2o0&quot;, &quot;completo
    &quot;zh&quot;: {&quot;express&quot;: &quot;price_1PZ9XBL0v2o0&quot;, &quot;completo
    &quot;es&quot;: {&quot;express&quot;: &quot;price_1PZ9XCL0v2o0&quot;, &quot;completo
    &quot;fr&quot;: {&quot;express&quot;: &quot;price_1PZ9XDL0v2o0&quot;, &quot;completo
    &quot;he&quot;: {&quot;express&quot;: &quot;price_1PZ9XEL0v2o0&quot;, &quot;completo
    &quot;hi&quot;: {&quot;express&quot;: &quot;price_1PZ9XFL0v2o0&quot;, &quot;completo
    &quot;en&quot;: {&quot;express&quot;: &quot;price_1PZ9XGL0v2o0&quot;, &quot;completo
    &quot;it&quot;: {&quot;express&quot;: &quot;price_1PZ9XHL0v2o0&quot;, &quot;completo
    &quot;ja&quot;: {&quot;express&quot;: &quot;price_1PZ9XIL0v2o0&quot;, &quot;completo
    &quot;pt&quot;: {&quot;express&quot;: &quot;price_1PZ9XJL0v2o0&quot;, &quot;completo
    &quot;ru&quot;: {&quot;express&quot;: &quot;price_1PZ9XKL0v2o0&quot;, &quot;completo
}

# Nomes dos Produtos Traduzidos
PRODUCT_NAMES = {
    &quot;pt&quot;: {&quot;express&quot;: &quot;Mapa Express&quot;, &quot;completo&quot;
    &quot;en&quot;: {&quot;express&quot;: &quot;Express Map&quot;, &quot;completo&quot;:
    &quot;es&quot;: {&quot;express&quot;: &quot;Mapa Express&quot;, &quot;completo&quot;
    # ... demais idiomas seguem o padrão
}

#
# 4. CRIAÇÃO DO APP (ORDEM CRÍTICA)
#
app = FastAPI(title=&quot;Global Numerology&quot;)

# Montagem de arquivos estáticos
STATIC_DIR = os.path.join(os.path.dirname(__file__), &quot;static&quot;)
if os.path.isdir(STATIC_DIR):
    app.mount(&quot;/static&quot;, StaticFiles(directory=STATIC_DIR), name=&quot;static&

#
# 5. MODELOS PYDANTIC
#
class PayReq(BaseModel):&lt;br/&gt;
    nome: str&lt;br/&gt;
    nascimento: str&lt;br/&gt;
    email: str&lt;br/&gt;
    lang: str = &quot;pt&quot;

class UrnaPayReq(BaseModel):&lt;br/&gt;
    nome_completo: str&lt;br/&gt;
    nome_urna: str&lt;br/&gt;
    email: str&lt;br/&gt;
```

```
    lang: str = &quot;pt&quot;

class EleitoralPayReq(BaseModel):&lt;br/&gt;
    nome_completo: str&lt;br/&gt;
    numero: str&lt;br/&gt;
    email: str&lt;br/&gt;
    lang: str = &quot;pt&quot;

class SugestaoReq(BaseModel):&lt;br/&gt;
    nome: str&lt;br/&gt;
    email: str&lt;br/&gt;
    mensagem: str

class BonusReq(BaseModel):&lt;br/&gt;
    nome: str&lt;br/&gt;
    email: str&lt;br/&gt;
    motivo: str

class AtivarBonusReq(BaseModel):&lt;br/&gt;
    codigo: str

class BannerContrato(BaseModel):&lt;br/&gt;
    id: str&lt;br/&gt;
    marca: str&lt;br/&gt;
    url_anunciante: str&lt;br/&gt;
    imagem_url: str&lt;br/&gt;
    escopo: str&lt;br/&gt;
    pais: str = &quot;&quot;&lt;br/&gt;
    continente: str = &quot;&quot;&lt;br/&gt;
    posicao: str = &quot;topo&quot;&lt;br/&gt;
    tipo: str = &quot;fixo&quot;&lt;br/&gt;
    data_inicio: str = &quot;&quot;&lt;br/&gt;
    data_fim: str = &quot;&quot;&lt;br/&gt;
    ativo: bool = True

#
# 6. FUNÇÕES DE CÁLCULO E UTILITÁRIOS
#
def reduzir(n):&lt;br/&gt;
    while n &gt; 9 and n not in [11, 22]:
        n = sum(int(d) for d in str(n))
    return n

def calc_mapa(nome, nascimento):
    # Lógica simplificada de numerologia
    d, m, a = map(int, nascimento.split(&#39;-&#39;))
    destino = reduzir(sum(map(int, str(d)+str(m)+str(a))))
    return {&quot;destino&quot;: destino}

def _enviar_email(para, assunto, corpo):&lt;br/&gt;
    try:
```

```
        msg = MIMEMultipart()
        msg[&#39;From&#39;] = FROM_EMAIL
        msg[&#39;To&#39;] = para
        msg[&#39;Subject&#39;] = assunto
        msg.attach(MIMEText(corpo, &#39;plain&#39;))
        # Configuração SMTP (Exemplo SendGrid/Gmail)
        with smtplib.SMTP(&quot;smtp.sendgrid.net&quot;, 587) as server:
            server.starttls()
            server.login(&quot;apikey&quot;, SENDGRID_KEY)
            server.send_message(msg)
        return True
    except Exception as e:&lt;br/&gt;
        logger.error(f&quot;Erro email: {e}&quot;)
        return False

#
# 7. ROTAS PRINCIPAIS
#
@app.get(&quot;/&quot;)
async def index():
    return RedirectResponse(url=&quot;/static/index.html&quot;)

@app.get(&quot;/config&quot;)
async def get_config():&lt;br/&gt;
    return {&quot;stripe_pub&quot;: STRIPE_PUB}

async def criar_checkout(price_id, email, metadata):&lt;br/&gt;
    try:
        session = stripe.checkout.Session.create(
            payment_method_types=[&#39;card&#39;],
            line_items=[{&#39;price&#39;: price_id, &#39;quantity&#39;: 1}],
            mode=&#39;payment&#39;,
            customer_email=email,
            success_url=SITE_URL + &quot;/static/sucesso.html?session_id={CHECKOUT_SESSI
            cancel_url=SITE_URL + &quot;/static/cancelado.html&quot;,
            metadata=metadata
        )
        return {&quot;id&quot;: session.id}&lt;br/&gt;
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))

@app.post(&quot;/pay/express&quot;)
async def pay_express(req: PayReq):
    p_id = PRICES.get(req.lang, PRICES[&quot;en&quot;])[&quot;express&quot;]
    return await criar_checkout(p_id, req.email, {&quot;tipo&quot;: &quot;express&quot;,

@app.post(&quot;/pay/complete&quot;)
async def pay_complete(req: PayReq):
    p_id = PRICES.get(req.lang, PRICES[&quot;en&quot;])[&quot;completo&quot;]
    return await criar_checkout(p_id, req.email, {&quot;tipo&quot;: &quot;completo&quot;
```

```
@app.post(&quot;/stripe_webhook&quot;)
async def stripe_webhook(request: Request):
    payload = await request.body()
    sig_header = request.headers.get(&quot;stripe-signature&quot;)
    try:
        event = stripe.Webhook.construct_event(payload, sig_header, os.getenv(&quot;STRI
    except Exception as e:&lt;br/&gt;
        return JSONResponse(content={&quot;error&quot;: str(e)}, status_code=400)

    if event[&quot;type&quot;] == &quot;checkout.session.completed&quot;:
        session = event[&quot;data&quot;][&quot;object&quot;]
        logger.info(f&quot;Pagamento confirmado: {session[&#39;id&#39;]}&quot;)
        # Lógica de geração de PDF e envio de email aqui
    return {&quot;status&quot;: &quot;success&quot;}

#
# 8. SISTEMA DE SUGESTÕES E BÔNUS
#
ARQ_BONUS = &quot;bonus_codes.json&quot;

def _carregar_codigos():&lt;br/&gt;
    if not os.path.exists(ARQ_BONUS): return {}&lt;br/&gt;
    with open(ARQ_BONUS, &quot;r&quot;) as f: return json.load(f)

def _salvar_codigos(dados):&lt;br/&gt;
    with open(ARQ_BONUS, &quot;w&quot;) as f: json.dump(dados, f, indent=2)

def _gerar_codigo_bonus():
    chars = string.ascii_uppercase + string.digits
    return f&quot;A1-{ &#39;&#39;.join(secrets.choice(chars) for _ in range(4)) }-{ &#39

PRODUTO_TARGET = {
    &quot;express&quot;:&quot;mapa&quot;, &quot;completo&quot;:&quot;mapa&quot;, &quot;i
}

@app.post(&quot;/sugestao&quot;)
async def receber_sugestao(req: SugestaoReq):&lt;br/&gt;
    corpo = f&quot;Sugestão de {req.nome} ({req.email}):

{req.mensagem}&quot;
    _enviar_email(ADMIN_EMAIL, &quot;Nova Sugestão A1ELOS&quot;, corpo)
    return {&quot;ok&quot;: True}

@app.post(&quot;/bonus&quot;)
async def solicitar_bonus(req: BonusReq):&lt;br/&gt;
    corpo = f&quot;Solicitação de Bônus:&lt;br/&gt;
Nome: {req.nome}&lt;br/&gt;
Email: {req.email}&lt;br/&gt;
Motivo: {req.motivo}&quot;
    _enviar_email(ADMIN_EMAIL, &quot;Solicitação de Bônus A1ELOS&quot;, corpo)
    return {&quot;ok&quot;: True}
```

```
@app.post(&quot;/ativar-bonus&quot;)
async def ativar_bonus(req: AtivarBonusReq):
    codigos = _carregar_codigos()
    info = codigos.get(req.codigo)
    if not info or info.get(&quot;usado&quot;):&lt;br/&gt;
        return {&quot;ok&quot;: False, &quot;msg&quot;: &quot;Inválido&quot;}
    info[&quot;usado&quot;] = True
    info[&quot;data_uso&quot;] = datetime.now().isoformat()
    _salvar_codigos(codigos)
    return {&quot;ok&quot;: True, &quot;target&quot;: PRODUTO_TARGET.get(info.get(&quot;

@app.post(&quot;/gerar-codigos-coletivo&quot;)
async def gerar_codigos_coletivo(request: Request):
    corpo = await request.json()
    itens = corpo.get(&quot;itens&quot;, [])
    codigos = _carregar_codigos()
    gerados = []
    for item in itens:&lt;br/&gt;
        for _ in range(item.get(&quot;qtd&quot;, 0)):
            cod = _gerar_codigo_bonus()
            codigos[cod] = {&quot;produto&quot;: item[&quot;produto&quot;], &quot;usado&
            gerados.append({&quot;codigo&quot;: cod, &quot;produto&quot;: item[&quot;pro
    _salvar_codigos(codigos)
    return {&quot;ok&quot;: True, &quot;codigos&quot;: gerados}

#
# 9. SISTEMA DE PUBLICIDADE GEOLOCALIZADA
#
ARQ_BANNERS = &quot;banners.json&quot;
PAIS_CONTINENTE = {&quot;BR&quot;:&quot;SA&quot;, &quot;US&quot;:&quot;NA&quot;, &quot;P

def _carregar_banners():&lt;br/&gt;
    if not os.path.exists(ARQ_BANNERS): return []&lt;br/&gt;
    with open(ARQ_BANNERS, &quot;r&quot;) as f: return json.load(f)

@app.get(&quot;/api/banner&quot;)
async def get_banner(posicao: str = &quot;topo&quot;, pais: str = &quot;BR&quot;):
    banners = _carregar_banners()
    continente = PAIS_CONTINENTE.get(pais.upper(), &quot;&quot;)
    for b in banners:&lt;br/&gt;
        if not b.get(&quot;ativo&quot;) or b.get(&quot;posicao&quot;) != posicao: contin
        if b.get(&quot;escopo&quot;) == &quot;pais&quot; and b.get(&quot;pais&quot;) ==
        if b.get(&quot;escopo&quot;) == &quot;continente&quot; and b.get(&quot;continent
        if b.get(&quot;escopo&quot;) == &quot;mundo&quot;: return {&quot;ok&quot;: True,
    return {&quot;ok&quot;: False, &quot;banner&quot;: None}

if __name__ == &quot;__main__&quot;:
    import uvicorn
    uvicorn.run(app, host=&quot;0.0.0.0&quot;, port=int(os.environ.get(&quot;PORT&quot;,
```

Documento gerado para suporte técnico A1ELOS. As chaves de API devem ser configuradas como variáveis de ambiente no Render.

REGISTRO DE MANUTENÇÃO — 14 de outubro de 2026 (3º registro)

Item: Correções 4 e 5 do GUIA_DE_CORRECOES_A1ELOS (main.py)

Status das duas correções: CONFIRMADAS COMO APLICADAS.

Correção 4 (criar função energia_texto): verificada no código atual do main.py. A função `energia_texto(texto)` está presente, com a lógica pitagórica correta (A=1 a I=9, J=1 a R=9, S=1 a Z=8), reduzindo a soma a 1-9 e preservando os números-mestre 11, 22 e 33; retorna None para textos sem letras válidas. Observação: a função está definida após gerar_sugestao_ia, o que é válido em Python (a resolução do nome só ocorre na chamada em tempo de execução).

Correção 5 (gerar_sugestao_ia): verificada. A função usa energia_texto corretamente e possui bases de nomes para os 11 produtos com sugestão de IA (nome_pet, nickname, nome_dominio, nome_canal, nome_equipe, nome_projeto, nome_evento, artistico, bebe, assinatura, negocio), com fallback para ["A1ELOS"].

Confirmação adicional — PRODUTOS_IA: o conjunto `PRODUTOS_IA` está com os 7 produtos corretos (pet, dominio, canal, equipe, projeto, evento, nickname), sem nome_ong, conforme a decisão de negócio de retirar o Nome de ONG da lista da Pesquisa IA.

Progresso do guia: 5 de 9 correções concluídas (1, 2, 3, 4 e 5). Pendências restantes: Correção 6 (fechar a lista mercado_rh_vantagens com a linha], em apresentacao_textos.py), Correção 7 (tradução de compatibilidade_equipes nos 14 idiomas), Correção 8 (PRODUTO_FAIXA com ia:0 e nome_ong:3), Correção 9 (nome da planilha com espaços em criar_precos_stripe.py) e a tarefa adicional de completar as chaves ar e he nos dicionários do frontend (ENERGIA_TITULOS, MONTAR_TRAD, DADO_LABEL, nomes de produtos).

RELATÓRIO DE SEGURANÇA E FECHAMENTO DO DIA — 14 de outubro de 2026 (5º registro)

STATUS: Console zerado, logs limpos. Bloco 1 (produtos.js) ENCERRADO com sucesso.

1. Resumo do dia — o que foi executado e verificado

Bloco 1 (produtos.js) — CONCLUÍDO, 9 sub-blocos:

- **1.1** — **PRODUTO_FAIXA** substituído integralmente: **24 produtos**, **ia:0**, **nome_ong:3**, **compatibilidade_equipes:5**. ✅
- **1.2** — **PRODUTOS_TRAD**: **24º produto** inserido nos **14 idiomas** (nomes he/ar vindos do **main.py**, fonte autorizada). ✅
- **1.3** — **ENERGIA_IDEAL**: **compatibilidade_equipes:8** inserido. ✅
- **1.4** — **BC_PRODUTOS**: linha do **24º produto** no grupo `R$ 98`. ✅
- **1.5** — **DADO_LABEL**: **compatibilidade_equipes** (Membros da equipe) nos **14 idiomas**, incluindo **he** (□□□□□ □□□□) e **ar** (□□□□□□ □□□□□). ✅
- **1.6** — **montarTabelaBC** reconstruída com corpo dentro do *try/catch* + correção da chave duplicada `p:` no *rotulos.ar* (agora `q:'□□□□□□□'`). ✅
- **1.7** — Remoção da duplicação no bloco **ru** do **PRODUTOS_TRAD** (*compatibilidade_equipes* em indonésio sobrescrevendo o russo). ⚠️ *Aplicar se ainda pendente.*
- **1.8** — **atualizarResumoBC** substituída (*aplicarEstadoBC* fora do loop, com *typeof* de proteção). ✅
- **1.9** — **FEAT_TRAD**: linha quebrada do **compatibilidade_equipes** corrigida para formato de array nos **14 idiomas** — ESTA era a causa raiz do *SyntaxError produtos.js:707*. ✅ Console zerado após a correção.

Causa raiz documentada: chave **compatibilidade_equipes** no **FEAT_TRAD** colada como string solta em vez de array, gerando sintaxe inválida que derrubava o arquivo inteiro no parser (*montarTabelaBC*, *montarEnergias* e *traduzirTudo* NÃO encontradas).

2. Pendências confirmadas para amanhã (15 de outubro)

- **Manhã — Bloco 2: translations.js** — chaves *p_compatibilidade_equipes* e *desc_compatibilidade_equipes* nos **14 idiomas** (resolve o **A5**: card RH não traduz, aparece em pt).
- **Tarde — Bloco 3: produtos.html** — resolver IDs duplicados *equipeEnergiaLabel/Dica/Sel* entre os cards **8** e **23** (**A4 crítico**: seletor do card **23** nunca recebe energia).
- **Bloco 4: main.py** — *IDEAL_ENERGIA* com *compatibilidade_equipes:"8"* (**A6**) + conferir *PRODUCT_NAMES* e *PRICE_IDS* do **24º produto** nos **14 idiomas**.
- **Bloco 5: apresentacao_textos.py** — *sobre_kpis* "23 Produtos Ativos" → "24" nos **14 idiomas** (**A7**).
- **Depois dos blocos: Correção 9** (*criar_precos_stripe.py* — nome da planilha com espaços) e rodada de testes de checkout por faixa de preço.

3. Checklist de preservação (condições mínimas mantidas)

- ✅ Nenhum trabalho anterior anulado — todas as correções **1 a 6** do guia preservadas e confirmadas no cruzamento.
- ✅ **PRODUTOS_IA** permanece com **7 produtos**, *nome_ong* fora da lista da IA.

- ✅ Ordem visual dos cards por preço preservada (*PRODUTO_FAIXA* não altera layout).
- ✅ Estrutura de **24 produtos** e **14 idiomas** intacta nos **5 arquivos**.